NERD - BD – Etapa 2

Pipeline de operadores no modelo materializado: SCAN, FILTER, PROJECT e AGGREGATE

Rafael Lima Dias
Projeto de extensão NERD - BD (Banco de Dados)

Novembro de 2026

Resumo

Este relatório descreve a etapa 2 do motor de consultas didático do Grupo A, escrito em C11 e baseado no modelo de execução materializado. Nesse modelo, cada operador consome a relação de entrada inteira e produz uma tabela de saída completa e independente antes que o operador seguinte comece. São apresentados o pipeline SCAN → FILTER → PROJECT → AGGREGATE, a projeção de colunas, as agregações `COUNT`, `SUM`, `MIN`, `MAX` e `AVG`, as regras de alocação e liberação de memória e a validação com o Valgrind. A execução registrada da suíte de testes terminou com 81 alocações, 81 liberações, nenhum vazamento e nenhum erro. O relatório também traz a arquitetura, o fluxograma, as dificuldades encontradas e possíveis otimizações.

1 Introdução

Um plano de consulta pode ser executado de duas formas principais. No modelo *pipeline* (ou iterador), as tuplas passam de um operador para o outro à medida que são produzidas. No modelo *materializado*, cada operador executa até o fim e grava todo o seu resultado em uma relação temporária, que é então lida pelo operador seguinte [1, 2]. O Grupo A implementa o segundo modelo.

A implementação não possui parser SQL. A consulta é montada no próprio código, encadeando chamadas de função e passando os nomes das colunas como argumentos. A consulta de referência desta etapa é

```
SELECT AVG(salary)
FROM (SELECT id, salary FROM funcionarios WHERE salary > 5000);
```

que corresponde ao exemplo do enunciado `SELECT id, salario WHERE salario > 5000`, seguido de uma agregação. No código as colunas têm nomes em inglês (`id`, `department`, `salary`, `age`). O cabeçalho do CSV, em português (`id,departamento,salario,idade`), é ignorado na leitura, e os campos são interpretados pela ordem do esquema.

2 Arquitetura

2.1 Módulos

A Tabela 1 resume a organização do código. Os arquivos `src/join.c` e `src/group_by.c` pertencem à etapa 3 e não são tratados aqui.

Arquivo	Responsabilidade
`include/table.h`, `src/table.c`	Modelo de dados (`Schema`, `Value`, `Row`, `Table`), vetor dinâmico de linhas e leitura de CSV tipada pelo esquema
`include/materialized.h`	API dos operadores, `QueryMetrics`, `Comparison`, `AggregateFunction`, `AggregateResult`
`src/scan.c`	`materialized_scan`
`src/materialized.c`	`materialized_filter`, `materialized_filter_salary_gt`, `materialized_project`, `materialized_project_indices`
`src/aggregate.c`	`materialized_aggregate`
`tests/test_materialized.c`	Testes unitários e o pipeline completo (`test_complete_pipeline`)
`Makefile`	Alvos `all`, `test`, `sanitize`, `valgrind`, `bench`, `clean`

Tabela 1: Módulos usados na etapa 2.

2.2 Estruturas de dados

Uma `Table` reúne um `Schema` e um vetor dinâmico de `Row`. O esquema guarda nome e tipo (`INT`, `DOUBLE` ou `STRING`) de até 16 colunas (`TABLE_MAX_COLUMNS`). Cada `Value` carrega o tipo e uma união com o valor; strings ficam dentro do próprio `Value`, em um vetor de 128 bytes, e nunca apontam para o buffer de leitura do CSV:

```c
/* Values own their string bytes; no row points into a CSV input buffer. */
typedef struct {
    DataType type;
    union {
        int int_value;
        double double_value;
        char string_value[VALUE_STRING_MAX_LEN];
    } as;
} Value;

typedef struct {
    Value values[TABLE_MAX_COLUMNS];
} Row;

typedef struct {
    Schema schema;
    Row *rows;
    size_t size;
    size_t capacity;
} Table;
```

Na plataforma de teste (x86-64, GCC), `sizeof(Value)` vale 136 bytes e `sizeof(Row)` vale $16 \times 136 = 2176$ bytes, independentemente de quantas colunas o esquema usa. Como o esquema é variável por tabela, a mesma estrutura representa a tabela de funcionários com 4 colunas e a tabela projetada com 2.

2.3 Contrato dos operadores

Todos os operadores seguem o mesmo contrato:

- recebem a entrada como `const Table *input` (somente leitura) e escrevem em `Table *output`;
- retornam `1` em sucesso e `0` em erro (ponteiro nulo, `input == output`, coluna inexistente, tipo incompatível ou falha de alocação);
- se falharem depois de começar a preencher a saída, chamam `table_free(output)` para liberar a saída parcial;
- quando `metrics` não é nulo, preenchem a estrutura `QueryMetrics`, com os campos `rows_in`, `rows_out`, `memory_bytes` e `time_ms`. O tempo vem de `clock()`, ou seja, é tempo de CPU.

A Tabela 2 lista os operadores da etapa.

Operador	Função	Comportamento
SCAN	`materialized_scan`	Cria a saída com o esquema da entrada e copia todas as linhas com `table_append`
FILTER	`materialized_filter`	Localiza a coluna, exige que o tipo do valor seja o da coluna, avalia `=`, `!=`, `<`, `<=`, `>` ou `>=` e copia as linhas aprovadas
FILTER (atalho)	`materialized_filter_salary_gt`	Chama `materialized_filter` com `salary > limite`
PROJECT	`materialized_project`	Monta um novo esquema com as colunas pedidas, na ordem pedida, e copia só esses valores
PROJECT (índices)	`materialized_project_indices`	Igual ao anterior, recebendo índices validados
AGGREGATE	`materialized_aggregate`	Percorre todas as linhas e devolve um único `AggregateResult`; não aloca memória de heap

Tabela 2: Operadores da etapa 2 (`include/materialized.h`).

3 Pipeline materializado

O encadeamento da consulta de referência, como aparece no teste `test_complete_pipeline` e no programa de demonstração, é o seguinte:

```
const char *columns[] = { "id", "salary" };
Table source, scanned, filtered, projected;
AggregateResult result;
table_init(&source);
assert(table_load_csv(&source, TEST_CSV_PATH));
assert(materialized_scan(&source, &scanned, NULL));
assert(materialized_filter_salary_gt(&scanned, 5000.0, &filtered, NULL));
assert(materialized_project(&filtered, columns, 2, &projected, NULL));
assert(materialized_aggregate(&projected, AGG_AVG, "salary", &result, NULL));
assert(projected.size == 3 && result.count == 3);
assert(fabs(result.value - (22800.0 / 3.0)) < 0.001);
table_free(&projected);
table_free(&filtered);
table_free(&scanned);
table_free(&source);
```

Cada operador recebe como entrada a tabela produzida pelo anterior e só começa depois que ela está completa. O SCAN, portanto, não devolve uma visão da tabela de origem, e sim uma cópia: o teste `test_scan_is_independent_materialization` altera a origem depois do SCAN e confirma que a saída não muda. A Tabela 3 mostra o efeito de cada passo sobre o arquivo `data/funcionarios.csv`, que tem 6 linhas.

Passo	Chamada	Entrada → saída
0	`table_load_csv`	CSV → `src` (6 linhas, 4 colunas)
1	`materialized_scan`	6 → 6 linhas (cópia independente)
2	`materialized_filter` (`salary > 5000.0`)	6 → 3 linhas (ids 2, 4 e 6)
3	`materialized_project` (`id, salary`)	3 linhas × 4 colunas → 3 linhas × 2 colunas
4	`materialized_aggregate`	3 linhas → 1 valor
5	`table_free` (ordem inversa)	0 bytes em uso ao final

Tabela 3: Fluxo de dados da consulta de referência.

O funcionário de id 3 tem salário exatamente 5000,00 e fica fora do resultado, pois a comparação é estrita (`COMPARE_GREATER`). O FILTER também é sensível a tipo: como `salary` é `DOUBLE`, o limite precisa ser passado como `value_double(5000.0)`; um `value_int(5000)` é rejeitado, e há teste para esse caso.

4 Projeção

A projeção constrói um novo esquema apenas com as colunas solicitadas e copia, para cada linha, os valores correspondentes. Seleção vazia, coluna inexistente ou mais de 16 colunas retornam erro. O trecho central de `materialized_project` é:

```
schema_init(&schema);
for (j = 0; j < column_count; ++j) {
    if (!schema_find_column(&input->schema, column_names[j], &indexes[j]) ||
        !schema_add_column(&schema, input->schema.columns[indexes[j]].name,
                           input->schema.columns[indexes[j]].type)) return 0;
}
table_init_with_schema(output, schema);
for (i = 0; i < input->size; ++i) {
    Row row;
    row_init(&row, &schema);
    for (j = 0; j < column_count; ++j) row.values[j] = input->rows[i].values[indexes[j]];
    if (!table_append(output, &row)) {
        table_free(output);
        return 0;
    }
}
```

Para a consulta `SELECT id, salary WHERE salary > 5000`, o programa de demonstração (`apoio/demo_etapa2.c`) produz a saída abaixo. Ele apenas chama a API e imprime cada tabela intermediária.

```
== FILTER salary > 5000 (rows=3, capacity=16, memory_bytes=34816)
   metrics: rows_in=6 rows_out=3 memory_bytes=34816
   id           department   salary       age
   2            finance      7200.00      41
   4            operations   9100.00      38
   6            engineering  6500.00      31

== PROJECT id, salary (rows=3, capacity=16, memory_bytes=34816)
   metrics: rows_in=3 rows_out=3 memory_bytes=34816
   id           salary
   2            7200.00
   4            9100.00
   6            6500.00
```

O endereço do vetor e o tempo de cada operador foram omitidos da saída acima, porque variam a cada execução. A tabela projetada ocupa os mesmos 34 816 bytes da filtrada (16×2176), já que a capacidade inicial é de 16 linhas e cada `Row` tem tamanho fixo.

5 Agregações

`materialized_aggregate` implementa `COUNT`, `SUM`, `MIN`, `MAX` e `AVG` sobre uma coluna numérica (`INT` ou `DOUBLE`); `COUNT` não precisa de coluna. O laço principal é:

```
for (i = 0; i < input->size; ++i) {
    double value = value_as_double(&input->rows[i].values[column_index]);
    sum += value;
    if (i == 0 || (function == AGG_MIN && value < result->value) ||
        (function == AGG_MAX && value > result->value)) result->value = value;
}
if (function == AGG_SUM) result->value = sum;
if (function == AGG_AVG && input->size != 0) {
    result->value = sum / (double)input->size;
    result->has_value = 1;
}
```

Em tabela vazia, `COUNT = 0` e `SUM = 0` são valores definidos, enquanto `MIN`, `MAX` e `AVG` retornam `has_value = 0`, o que evita divisão por zero e valores fictícios. Aplicadas à tabela projetada, as cinco agregações produzem:

```
== AGGREGATE sobre a tabela projetada
   COUNT = 3.00 (count=3, has_value=1)
   SUM   = 22800.00 (count=3, has_value=1)
   MIN   = 6500.00 (count=3, has_value=1)
   MAX   = 9100.00 (count=3, has_value=1)
   AVG   = 7600.00 (count=3, has_value=1)
```

Os valores conferem com os dados: $7200 + 9100 + 6500 = 22\,800$ e $22\,800/3 = 7600$.

6 Gerenciamento de memória

6.1 Quem aloca e quem libera

A única memória de heap da etapa 2 é o vetor `Table.rows`. Como as strings ficam embutidas em `Value`, não há `malloc` por célula nem por string. Quem cria uma `Table` é dono do seu vetor e deve chamar `table_free` depois que o último consumidor terminar (Tabela 4).

Recurso	Quem aloca	Quem libera
`rows` da tabela fonte	`table_load_csv`, via `table_append`	o chamador, com `table_free`
`rows` da saída de SCAN, FILTER e PROJECT	o operador, via `table_append`	o chamador, depois que o operador seguinte termina
`AggregateResult`	variável do chamador (pilha)	nada a liberar
`Row` temporária (PROJECT, CSV)	pilha da função	fim do escopo; `table_append` copia por valor
Strings de célula	parte do `Value`	junto com `rows`
Saída parcial em caso de erro	o operador	o próprio operador, antes de retornar `0`
`FILE *` do CSV	`fopen` em `table_load_csv`	`fclose` em todos os caminhos

Tabela 4: Propriedade da memória na etapa 2.

6.2 Crescimento do vetor e reutilização de buffers

O vetor de linhas começa com capacidade 16 e dobra a cada estouro, por meio de `realloc`, com verificação de overflow no cálculo do tamanho:

```
static int table_reserve(Table *table, size_t capacity)
{
    Row *rows;

    if (capacity > (size_t)-1 / sizeof(*rows)) return 0;
    rows = realloc(table->rows, capacity * sizeof(*rows));
    if (rows == NULL) return 0;
    table->rows = rows;
    table->capacity = capacity;
    return 1;
}

static int table_grow(Table *table)
{
    size_t capacity = table->capacity == 0 ? INITIAL_CAPACITY : table->capacity * 2;
    if (capacity < table->capacity) return 0;
    return table_reserve(table, capacity);
}
```

O código não reutiliza buffers entre operadores: cada operador aloca uma `Table` nova. O único reaproveitamento é o do próprio vetor de uma tabela, que o `realloc` pode estender no lugar. Isso é coerente com o modelo materializado, em que a entrada precisa continuar válida enquanto o operador a lê. Depois que o consumidor termina, o chamador pode liberar o resultado intermediário. Como `memory_bytes` é calculado como `capacity` × `sizeof(Row)`, a métrica mede o buffer efetivamente reservado, que pode ser maior que o número de linhas.

6.3 Validade dos ponteiros

- `table->rows` e `&table->rows[i]` valem até o próximo `table_append` que precise crescer o vetor, porque o `realloc` pode movê-lo, ou até `table_free`. A API não guarda esses ponteiros entre operadores.
- Dentro de um operador, a entrada é lida e copiada por valor; nenhum ponteiro para ela sobrevive à chamada.
- Os nomes de coluna recebidos como `const char *` só precisam ser válidos durante a chamada, pois `schema_add_column` os copia para o novo esquema.
- `input == output` é rejeitado em SCAN, FILTER e PROJECT, para que o operador não destrua o buffer que está lendo.
- `table_free` aceita tabela vazia e zera `rows`, `size`, `capacity` e o esquema, então chamá-la duas vezes é seguro.
- Os operadores inicializam a saída com `table_init_with_schema` sem liberar o conteúdo anterior. A saída precisa chegar vazia; reutilizar uma tabela com linhas sem chamar `table_free` antes causaria vazamento.

6.4 Como os vazamentos são evitados

1. Há uma única fonte de heap (`rows`) e um único liberador (`table_free`).
2. Em qualquer falha no meio da materialização, o operador libera a saída parcial antes de retornar. Exemplo do FILTER:

```
if (matches && !table_append(output, &input->rows[i])) {
    table_free(output);
    return 0;
}
```

3. `table_load_csv` fecha o arquivo e libera a tabela se encontrar uma linha inválida.
4. As tabelas são liberadas na ordem inversa da criação (`projected`, `filtered`, `scanned`, `src`).

6.5 Validação com Valgrind

A suíte de testes foi executada sob o Memcheck do Valgrind [3] pelo alvo `make valgrind`:

```
valgrind --leak-check=full --show-leak-kinds=all --error-exitcode=1 ./test_materialized
```

O ambiente registrado foi Linux (Debian), GCC 14.2.0 e Valgrind 3.24.0. O resumo do Valgrind foi:

```
HEAP SUMMARY:
    in use at exit: 0 bytes in 0 blocks
  total heap usage: 81 allocs, 81 frees, 1,401,373 bytes allocated

All heap blocks were freed -- no leaks are possible

ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

A Tabela 5 reúne as três execuções registradas. A suíte `test_materialized` inclui também os testes da etapa 3, e o executável `miniquery` roda a consulta da etapa 3; o pipeline isolado da etapa 2 corresponde à linha `demo_etapa2`. Além do Valgrind, `make sanitize` (AddressSanitizer e UndefinedBehaviorSanitizer) terminou com `all materialized tests passed`.

Programa	Alocações	Liberações	Bytes alocados	Em uso no fim	Erros
`test_materialized`	81	81	1 401 373	0	0
`miniquery`	24	24	396 888	0	0
`demo_etapa2`	7	7	147 928	0	0

Tabela 5: Resultados do Valgrind Memcheck.

As 7 alocações do `demo_etapa2` correspondem ao modelo descrito: um vetor para cada uma das quatro tabelas (fonte, SCAN, FILTER e PROJECT), todas com capacidade 16 ($4 \times 34\,816 = 139\,264$ bytes), mais as alocações internas da biblioteca padrão para o arquivo CSV e a saída.

7 Fluxograma

A Figura 1 mostra o fluxo completo da etapa 2, incluindo os caminhos de erro e a ordem de liberação das tabelas.

8 Testes

`make test` compila a suíte com `-std=c11 -Wall -Wextra -Wpedantic -Werror -O2` e termina com `all materialized tests passed`. Para a etapa 2, os testes cobrem a leitura do CSV e o esquema, a independência da cópia do SCAN, os limites e erros do FILTER, a projeção ordenada e seus casos de erro, cada agregação (inclusive em tabela vazia) e o pipeline completo.

9 Dificuldades encontradas

- **Linha de tamanho fixo.** A estrutura anterior usava uma linha fixa de funcionário (`EmployeeRow`), o que impedia projetar colunas em outra ordem ou com esquema menor. Ela foi substituída por `Schema` e `Row`/`Value` tipados, com esquema variável por tabela, mantendo o formato do CSV.
- **Propriedade das strings.** Ponteiros para o buffer de leitura do CSV ficariam inválidos e strings alocadas por célula aumentariam o risco de vazamento. A solução foi embutir as strings no `Value`, ao custo de um limite de 127 caracteres por valor e de mais memória por linha.
- **Valor esperado no teste.** O valor esperado da média no teste do pipeline foi corrigido para $22\,800/3 = 7600$, que é a média dos três salários filtrados.
- **Ambiente de validação.** A validação de memória com Valgrind, AddressSanitizer e UBSan foi feita em Linux, com os resultados da Seção 6.5.
- **FILTER sensível a tipo.** Comparar `salary` (`DOUBLE`) com um inteiro é rejeitado. Isso foi mantido como decisão de projeto: o chamador usa `value_double(5000.0)`.

10 Possíveis otimizações

As otimizações a seguir partem de limitações reais do código atual.

1. **Liberar cada intermediário assim que o consumidor termina.** Hoje, no teste do pipeline, todas as tabelas ficam vivas até o fim, e o pico de memória é a soma de fonte, SCAN, FILTER e PROJECT. Liberando cada uma logo depois do operador seguinte, o pico cairia para cerca de duas tabelas.
2. **SCAN sem cópia ou fundido com o FILTER.** O SCAN duplica a tabela inteira. Eliminar essa cópia economiza memória, mas deixa de ser uma materialização pura.
3. **Linha compacta.** Cada `Row` reserva 16 valores de 136 bytes (2176 bytes), mesmo com 2 colunas. Alocando só as colunas do esquema, a tabela projetada `id, salary` usaria cerca de 32 bytes de dados por linha.
4. **Strings fora da linha.** Um pool ou arena de strings de tamanho variável reduziria a memória, desde que se defina claramente o dono e a liberação.
5. **Armazenamento colunar.** PROJECT e AGGREGATE leriam só as colunas usadas, com melhor aproveitamento de cache.

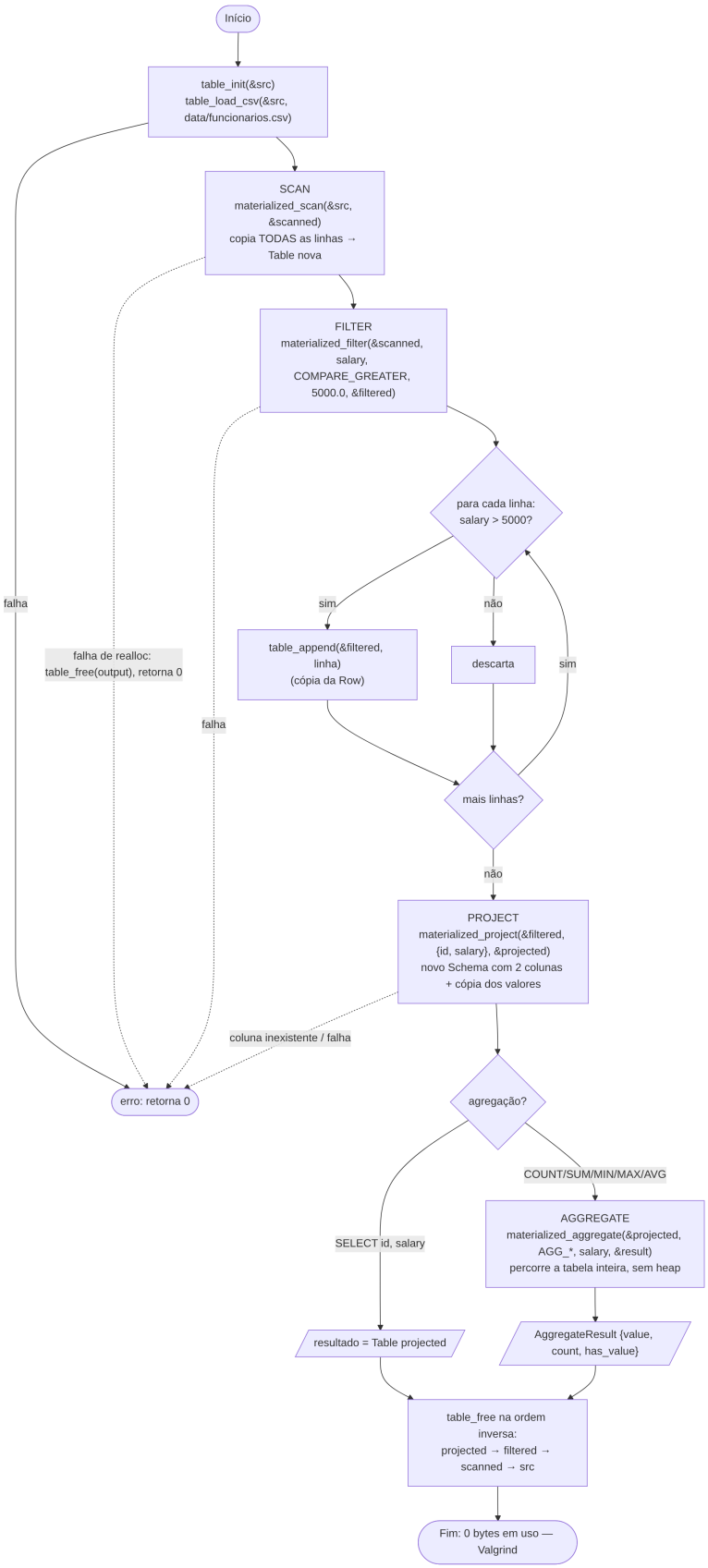


Figura 1: Fluxograma do pipeline materializado da etapa 2.

6. **Capacidade exata.** SCAN e PROJECT conhecem `input->size` de antemão e poderiam reservar o vetor uma única vez, sem `realloc` repetidos nem capacidade sobrando.
7. **Arena por consulta.** Todas as tabelas de uma consulta seriam liberadas de uma vez, com menos fragmentação.
8. **Fusão de FILTER e PROJECT e predicados compostos.** Menos passagens pelos dados, aproximando a execução do modelo pipeline [1].
9. **Medição de tempo de parede com repetições.** `clock()` mede tempo de CPU e fica perto de zero com 6 linhas. Benchmarks confiáveis precisam de várias execuções e da mediana.
10. **Leitor de CSV mais robusto.** `table_load_csv` ignora o cabeçalho e separa campos com `strtok`. Validar o cabeçalho contra o esquema e tratar aspas e valores nulos evitaria erros de dados.

11 Conclusão

A etapa 2 implementa o pipeline SCAN → FILTER → PROJECT → AGGREGATE no modelo materializado, com cada operador produzindo uma tabela própria e independente. A projeção `SELECT id, salary WHERE salary > 5000` retorna as 3 linhas esperadas, e as cinco agregações produzem valores coerentes com os dados. A posse da memória é simples: um único vetor de heap por tabela, liberado pelo chamador, com liberação da saída parcial em todo caminho de erro. A validação com Valgrind terminou com 81 alocações, 81 liberações, nenhum vazamento e nenhum erro.

Referências

[1] G. Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2):73–169, 1993.

[2] H. Garcia-Molina, J. D. Ullman e J. Widom. *Database Systems: The Complete Book*. 2. ed. Pearson Prentice Hall, 2008.

[3] Valgrind Developers. *Valgrind User Manual – Memcheck: a memory error detector*. Disponível em: https://valgrind.org/docs/manual/mc-manual.html.